Assignment no #01

Submitted by:

Laiba Mubeen

Roll No: 22011519-090

Submitted to: **Sir Zaheer**

Subject: **Compiler Counstructor**

Date: **5 Nov 2025**

Univeristy of Gujrat Sub Campus M.B.D

Question no # 01

What is the difference between a compiler and an interpreter?

Compiler

A compiler is a language processor that translates the entire program written in a high-level language into machine code (object code) before execution. It checks all errors together and then generates an executable file.

Example:

C program (compiled once by GCC or Turbo C++) generates .exe file can run many times without recompilation.

Interpreter

An interpreter translates one line (or statement) at a time of a high-level program into machine code and executes it immediately.

It does not produce a separate object or executable file.

Example:

Python program runs line by line each time you execute it → no .exe file generated.

Question no # 02

What are the advantages of

(a) a compiler over an interpreter

A compiler has several benefits compared to an interpreter because it translates the whole program at once and produces an executable file.

a. Faster Execution

Once a program is compiled, it becomes machine code and runs very fast.

Execution does not need translation again and again.

Example:

A compiled C program runs faster than a Python script.

b. Creates an Executable File

Compiler generates a .exe (or object) file which can be run anytime without recompilation

Interpreter cannot do this — it executes line by line every time.

c. Efficient for Large Program

Suitable for large and complex programs because compilation is done only once. Interpreter re-reads each line again, so it's slower for big projects.

d. Error Checking for Whole Program

Compiler checks entire code at once and gives a complete list of errors.

This helps the programmer fix all errors together

E.Protection of Source Code

Since compiler generates a separate executable file, the original source code remains hidden from users.

What are the advantages of

(b) A interpreter over a compiler?

An interpreter is more flexible and easier to debug, especially for learners and scripting languages.

1.Easy to Debug

Interpreter stops at the exact line where the error occurs, making debugging quick and simple.

You can fix one error and continue from the same point.

2.No Need for Compilation

Interpreter runs code directly, no need to wait for compilation.

Very useful for testing small code snippets quickly.

3.Platform Independent

Many interpreted languages (like Python, JavaScript) can run on any system where the interpreter is available — no need to recompile for each platform.

4.Less Memory Usage Initially

Since it executes line by line, interpreter doesn't store the full compiled code in memory.

This makes it lighter for small or educational programs.

5.Suitable for Beginners and Scripting

Interpreter-based languages are beginner-friendly because you can see results immediately.

Perfect for learning programming logic and testing small code pieces.

Question no # 03

What advantages are there to a language-processing system in which the compiler produces assembly language rather than machine language?

1.Easier to Understand and Debug

- Assembly language is human-readable (uses mnemonics like MOV, ADD, JMP) while machine language is just binary (0s and 1s).
- So, if the compiler produces assembly code, programmers can read, understand, and debug the output more easily.
- □ Example: It's easier to find where a compiler made a mistake by checking assembly instructions.

2. Easier for Portability and Modification

- Assembly code can be modified manually before converting to machine code.
- Developers can optimize or tune certain parts for better performance on different hardware

3. Simplifies Compiler Design

- Writing a compiler that translates to assembly is simpler than writing one that directly produces machine code.
- The assembler (next tool) takes care of converting assembly → machine code.
- □ So, the compiler's job becomes easier and more structured.

4. Use of Assembler for Final Conversion

- The assembler is a specialized program that converts assembly language into machine code.
- Since assemblers are already optimized and tested, using them ensures more accurate and efficient machine code generation.

5. Better Error Handling

- Because assembly is readable, it's easier to trace errors that occur during translation.
- The programmer can check intermediate code before final execution.

6. Useful for Multi-Stage Compilation

- In *some systems, compilation is done in stages:*
- *High-level language → Assembly → Machine code*
- Producing assembly code helps to reuse the assembler for multiple compilers (C, C++, FORTRAN, etc.), reducing effort.

Question no # 04

A compiler that translates a high-level language into another high-level language is called a source-to-source translator. What advantages are there to using C as a target language for a compiler?

A source-to-source compiler (or translator) is a type of compiler that converts a program written in one high-level programming language (HLL) into another high-level language, instead of directly producing machine code.

Example:

Translating a program written in Python-like language into C language. When such a compiler uses C as the target language, there are many advantages discussed below 👇

Advantages of Using C as a Target Language

1. High Portability

- C is a portable language, meaning C programs can be compiled on almost any operating system or hardware.
- If the compiler outputs C code, that program can easily be compiled with any standard C compiler (like GCC) on any machine.
- Example:

- If your custom language is converted to C, you can run it on Windows, Linux, or Mac easily by recompiling the generated C code.

2. Readable and Debuggable Output

- C language is human-readable, unlike assembly or machine code.
- This makes the translated program easier to understand, debug, and test.
- Developers can check if the compiler is generating correct logic.

3. Avoids Writing a Machine Code Generator

- Writing a compiler that directly produces machine code is complex and hardware-dependent.
- By using C as the target, the compiler skips that difficult part, because the C compiler will handle the conversion from C → machine code.

4. Reuse of Existing C Compiler and Optimizer

- Every system already has a highly optimized C compiler.
- So, by producing C code, we can reuse existing C compiler tools (like optimization, linking, debugging) instead of building new ones.

5. Easy Integration with Existing C Libraries

- Using C as target allows the translated program to use existing C libraries for input/output, math, or system functions.
- This saves development time and increases compatibility.

Question no # 05

Describe some of the tasks that an assembler needs to perform.

An Assembler is a language processor that translates a program written in Assembly Language into Machine Language (Object Code) that the computer can execute directly.

It performs several important tasks during this translation process.

Main Tasks Performed by an Assembler

1. Translation of Mnemonics to Machine Code

- The most important task of an assembler is to convert assembly mnemonics (like MOV, ADD, SUB) into machine instructions (binary code).
- Example:
- MOV AX, BX → converted into a binary code like 1000100111011000.

2. Symbol Table Creation

- Assembler maintains a symbol table that keeps track of labels, variables, and their memory addresses.
- This helps during both translation and linking processes.
- Example:
- If you write LOOP: ADD AX, 1, assembler stores the address of label LOOP in the symbol table.

3. Address Calculation (Location Counter)

- Assembler calculates the memory address of each instruction and variable.

- It uses a location counter to assign sequential addresses to instructions.

4. Handling of Data and Constants

- Assembler identifies data definitions (like DB, DW, etc.) and allocates appropriate memory space for them.
- It also replaces symbolic constants with their actual numeric values.

5. Error Detection

- Assembler checks for syntax errors such as:
- Undefined labels
- Wrong instruction format Invalid operands
- If found, it displays error messages to the programmer.

6. Generating Object Code

- After successful translation, assembler produces the object file (machine code) which is ready for execution or linking.
- □ Example:
- Assembly file (.asm) → Object file (.obj)

Question no # 06

Problem Statement (OBJECTIVE)

Design a lexical analyzer for given language and the lexical analyzer should ignore redundant spaces, tabs and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you

may restrict the length to some reasonable value. Simulate the same in C language.

RESOURCE:

Turbo C ++ PROGRAM LOGIC:

1. Read the input Expression.

2. Check whether input is alphabet or digits then store it as identifier.

3. If the input is operator store it as symbol. 4. Check the input for keywords

Code:

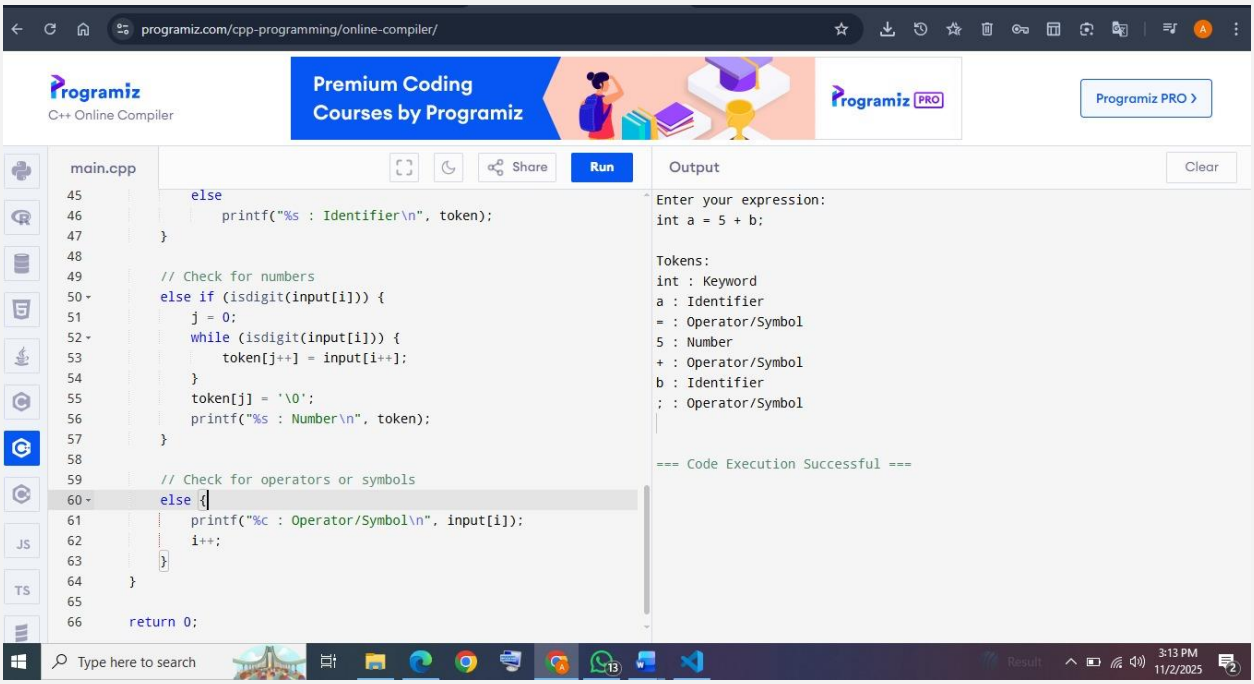
```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int isKeyword(char *word) {
   char keywords[][10] = {"int", "float", "if", "else", "while", "return", "for", "char"};
   for (int i = 0; i < 8; i++) {
      if (strcmp(word, keywords[i]) == 0)
         return 1;
   }
   return 0;
}
int main() {
```

```
char input[100], token[30];
int i = 0, j = 0;
printf("Enter your expression:\n");
fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';
printf("\nTokens:\n");
while (input[i] != '\0') {
    if (input[i] == ' ' || input[i] == '\t' || input[i] == '\n') {
        i++;
        continue;
    }
    if (input[i] == '/' && input[i + 1] == '/') break;
    if (isalpha(input[i])) {
        j = 0;
        while (isalnum(input[i])) {
            token[j++] = input[i++];
        }
        token[j] = '\0';
        if (isKeyword(token))
            printf("%s : Keyword\n", token);
        else
            printf("%s : Identifier\n", token);
```

```
        }
        else if (isdigit(input[i])) {
            j = 0;
            while (isdigit(input[i])) {
                token[j++] = input[i++];
            }
            token[j] = '\0';
            printf("%s : Number\n", token);
     }
        else {
            printf("%c : Operator/Symbol\n", input[i]);
            i++;
        }
    }

    return 0;
}
```

Screeshot:


programiz.com/cpp-programming/online-compiler/
Programiz
C++ Online Compiler
Premium Coding Courses by Programiz
Programiz PRO
Programiz PRO
main.cpp
Share
Run
Output
Clear
else
printf("%s : Identifier\n", token);
}
// Check for numbers
else if (isdigit(input[i])) {
j = 0;
while (isdigit(input[i])) {
token[j++] = input[i++];
}
token[j] = '\0';
printf("%s : Number\n", token);
}
// Check for operators or symbols
else {
printf("%c : Operator/Symbol\n", input[i]);
i++;
}
}
return 0;
Enter your expression:
int a = 5 + b;
Tokens:
int : Keyword
a : Identifier
= : Operator/Symbol
5 : Number
+ : Operator/Symbol
b : Identifier
; : Operator/Symbol
=== Code Execution Successful ===
Type here to search
3:13 PM
11/2/2025